

TODO 競賽名稱

TODO 競賽名稱 試題本

競賽規則

1. 競賽時間：202TODO/TODO/TODO TODO:TODO ~ TODO:TODO，共 TODO 小時。
2. 本次競賽試題共 TODO 題，每題皆有子任務。
3. 為了愛護地球，本次競賽題本僅提供電子檔，不提供紙本。
4. 每題的分數為該題所有子任務得分數加總；單筆子任務得分數為各筆繳交在該筆得到的最大分數。
5. 本次初選比照南區賽提供記分板，複選比照全國賽不提供記分板。
6. 全部題目的輸入皆為標準輸入。
7. 全部題目的輸出皆為標準輸出。
8. 所有輸入輸出請嚴格遵守題目要求，多或少的換行及空格皆有可能造成裁判系統判斷為答案錯誤。
9. 每題每次上傳間隔為 120 秒，裁判得視情況調整。
10. 所有試題相關問題請於競賽系統中提問，題目相關公告也會公告於競賽系統，請密切注意。
11. 如有電腦問題，請舉手向監考人員反映。
12. 如有如廁需求，須經過監考人員同意方可離場。
13. 不得攜帶任何參考資料，但競賽系統上的參考資料可自行閱讀。
14. 不得自行攜帶隨身碟，如需備份資料，請將資料儲存於電腦 D 槽。
15. 競賽中請勿交談。請勿做出任何會干擾競賽的行為。
16. 如需使用 C++ 的 `std::cin` 或 `std::cout` 可將以下程式碼插入 `main function` 以及將 `endl` 取代為 `'\n'` 來最佳化輸入輸出速度。唯須注意不可與 `cstdio` 混用。

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```

A. 彩虹遊行

Problem ID: Rainbow_Parade

Time Limit: 1s

Memory Limit: 512MiB

家豪（Charhao）身為台南一中的三好學生，平時最喜歡翹課出來參加彩虹遊行。顧名思義，彩虹遊行指的是一大群彩虹小馬愛好者聚在一起的盛大活動。

今天的彩虹遊行在一座神奇的島嶼上舉辦，島上有 n 個彩虹小馬景點，編號為 $1 \sim n$ 。島嶼的入口位於 1 號景點，而開幕式場地則在 n 號景點。

家豪對這個島嶼一無所知，一個人走一定會迷路，所以他需要熱心助人的彩虹小馬學長 tw87 帶路。島上共有 m 條單向道路，第 i 條道路從景點 a_i 連向景點 b_i 。

由於 tw87 趕著去參加「彩虹小馬聖經研討會」，無法親自帶路，於是他決定畫一張地圖給家豪。地圖的繪製規則如下：

- 除了開幕式景點 n 之外，每個景點都必須**恰好選擇**一條從該點出發的單向道路，寫下一個該去的景點。
- 為了避免地圖標示混亂，除了入口 1 號景點外，其餘每個景點在地圖上**恰好只能被一個景點指引**。
- 家豪必須能順利從入口 1 號景點沿著指引走到開幕式 n 號景點。但為了滿足 tw87 的惡趣味，對於**不在這條主線路徑上**的其他景點，tw87 刻意將它們設計成**若家豪不小心闖入，將會陷入無限繞圈圈的死循環中**。

每條道路 i 都有一個「奇怪值」 d_i 。只要 tw87 在地圖上記錄了該條道路，該道路的奇怪值就會累加到總和中。

tw87 想在滿足自己惡趣味的前提下，盡可能最小化總奇怪值，以免被可愛的學弟討厭。身為一中的彩虹新秀，請你幫 tw87 計算出最小可能的總奇怪值為多少？

— 輸入 —

第一行輸入二正整數 n m

接下來 m 行，輸入二正整數與一非負整數 a_i, b_i, d_i

— 輸出 —

輸出一個整數，代表最小可能的總奇怪值。

若無法構造出滿足條件的地圖，則輸出 -1。

— 輸入限制 —

- $2 \leq n \leq 2000$
- $1 \leq m \leq \min(10^4, n \times (n - 1))$
- $1 \leq a_i, b_i \leq n, a_i \neq b_i$
- $0 \leq d_i \leq 10^5$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	8	$2 \leq n \leq 8, 1 \leq m \leq \min(30, n \times (n - 1)), 0 \leq d_i \leq 100$
3	17	$2 \leq n \leq 18, 1 \leq m \leq \min(200, n \times (n - 1)), 0 \leq d_i \leq 100$
4	22	$2 \leq n \leq 1000, 1 \leq m \leq \min(5000, n \times (n - 1)), 0 \leq d_i \leq 1$
5	24	$2 \leq n \leq 200, 1 \leq m \leq \min(1000, n \times (n - 1)), 0 \leq d_i \leq 10^5$
6	29	無額外限制

— 範例輸入 —

4 6
1 2 5
1 3 1
2 3 2
2 4 2
3 2 3
3 4 4

— 範例輸出 —

6

B. TODO: 題目名稱

Problem ID: TODO:_English_short_name

Time Limit: 1s

Memory Limit: 512MiB

車輛來來去去的馬路上有一條斑馬線。

這條斑馬線的每個白色區塊（共 n 個）可以視為在一維數線上依序排列，位置依序處於 $x_1, x_2, \dots, x_n$ 。

今天有 m 隻小雞位於第一塊白色區塊（ x_1 ），想要到達最後一塊區塊（ x_n ）後，再折返回第一塊區塊。

然而，斑馬線上的白色區塊會因為小雞們的踩踏而磨損。除了第一塊與最後一塊區塊異常堅固（不會磨損）之外，對於中間的任何一個白色區塊 i ，只要有一隻小雞跳到上面，它的被踩踏次數就會加 1。當該區塊總共被踩踏了 a_i 次後，就會變成與路面一樣的顏色而消失（不再屬於斑馬線）。

小雞們是很遵守交通規則的，所以牠們無論去程或回程，都只會踩在還沒消失的白色區塊上。假設每隻小雞的跳躍能力相同，請你幫忙計算：為了讓 m 隻小雞都能順利完成往返目標，小雞們的「單次最大跳躍距離」至少需要是多少？

— 輸入 —

第一行輸入二正整數 n m

第二行輸入 n 個整數 $x_1, x_2, \dots, x_n$

第三行輸入 $n - 2$ 個正整數 $a_2, a_3, \dots, a_{n-1}$

— 輸出 —

輸出小雞們的「單次最大跳躍距離」至少需要是多少。

— 輸入限制 —

- $3 \leq n \leq 10^5$
- $1 \leq m \leq 10^5$
- $|x_i| \leq 10^9$
- $1 \leq a_i \leq 10^9$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	7	$m = 1, a_i = 10^9$
3	18	$3 \leq n \leq 2000, m = 1, a_i = 1$
4	9	$3 \leq n \leq 7, 1 \leq m \leq 7$
5	20	$3 \leq n \leq 100, 1 \leq m \leq 100, \sum a_i \leq 10^4$
6	22	$3 \leq n \leq 2000$
7	24	無額外限制

— 範例輸入 —

— 範例輸出 —

D. Merge, Rebase, Squash!

Problem ID: Git

Time Limit: 2.5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學 DB force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者，TOJ 整個程式碼由 H 個獨立的 **hunk** 組成，每個 hunk 的狀態一開始都是 0，在 v2.0 分支上面。

有天 wonderhoi 想給 TOJ 貢獻一個酷炫的功能，於是 wonderhoi 很高興的開啟了一個新的分支 feat/fancy 並在上面建立了 N 個 commit。

wonderhoi 想讓功能進到 TOJ 的 v2.0，於是請 tobiichi3227 來 review，tobiichi3227 覺得 N 個 commit 太多了，應該用 squash 把 commit 減少到只有 K 個。

當 squash 完成後，會將 feat/fancy rebase 到 v2.0，wonderhoi 發現會有 conflict，所以他想要最小化解決 conflict 的成本，但 wonderhoi 要去比地奧 (IESO) 沒空了，於是交給在看题目的你。

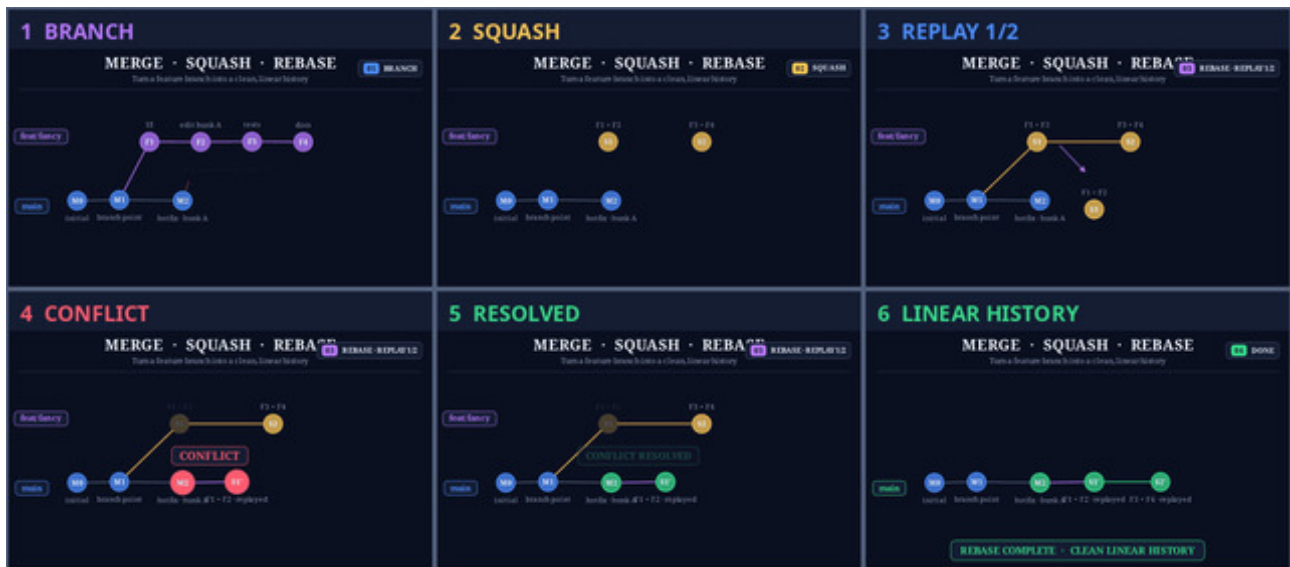


Figure 2: Branch、squash、rebase 與 conflict resolution 的執行過程

若使用支援 PDF 附件的 Firefox，可
分鏡。

；Chrome 與 Edge 請直接閱讀上方

— Branch 狀態 —

v2.0 branch 上第 h 個 hunk 的最終內容為 U_h 。

feat/fancy branch 上共有 N 個 commits。第 i 個 commit 會將第 x_i 個 hunk 的內容改成 v_i 。

令 $S_i[h]$ 表示執行完 feat/fancy branch 的前 i 個 commits 後，第 h 個 hunk 的內容。

初始時： $S_0[h] = 0$

第 i 個 commit 執行後： $S_i[x_i] = v_i$

其他 hunk 的內容不變。

– Squash –

原本的 N 個 commits 依照順序排列：

commit 1, commit 2, ..., commit N

你需要在這些 commits 之間放置 $K - 1$ 個切點，將它們分成恰好 K 組。

每一組中的所有 commits 會被合併成一個新的 **squash commit**。

例如，假設共有 7 個 commits，且要分成 3 組，可以選擇：

```
[commit 1, commit 2]
[commit 3, commit 4, commit 5]
[commit 6, commit 7]
```

也可以簡寫成：

```
[1, 2] [3, 4, 5] [6, 7]
```

這代表：- 第一個 squash commit 包含原本的 commits 1 到 2 - 第二個 squash commit 包含原本的 commits 3 到 5 - 第三個 squash commit 包含原本的 commits 6 到 7

為了描述一般情況，令： $0 = p_0 < p_1 < p_2 < \dots < p_K = N$

其中 p_g 表示第 g 組的最後一個 commit 編號。

因此，第 g 個 squash commit 包含： $p_{g-1} + 1, p_{g-1} + 2, \dots, p_g$

例如前面的分組：

```
[1, 2] [3, 4, 5] [6, 7]
```

對應： $p_0 = 0, p_1 = 2, p_2 = 5, p_3 = 7$

考慮第 g 個 squash commit，以及某個 hunk h 。

在這組 commits 開始以前，hunk h 的內容是： $S_{p_{g-1}}[h]$

執行完這組中的所有 commits 後，hunk h 的內容是： $S_{p_g}[h]$

因此，這個 squash commit 對 hunk h 的效果為： $S_{p_{g-1}}[h] \rightarrow S_{p_g}[h]$

也就是只保留這一組 commits 執行前與執行後的差異，中間經過的狀態不會出現在 squash commit 中。

例如，某個 hunk 在一組 commits 中依序發生：

1 -> 3 -> 5 -> 2

那麼 squash 後只會留下：

1 -> 2

中間的 3 與 5 都不會出現在 squash commit 中。

若一組 commits 執行前後，某個 hunk 的內容相同，即： $S_{p_{g-1}}[h] = S_{p_g}[h]$

則這個 squash commit 不會修改該 hunk。

– Rebase 與 Conflict –

完成 squash 後，tobiichi3227 會依照原本的順序，將這 K 個 squash commits 逐一套用到 v2.0 branch 上。

考慮其中一個 squash commit 對 hunk h 的修改： $\text{old} \rightarrow \text{new}$

其中：- old：這個 squash commit 預期修改前的內容；- new：這個 squash commit 想要修改成的內容；- current：目前 v2.0 branch 上實際存在的內容。

例如： $\text{old} = 3$, $\text{new} = 7$, $\text{current} = 5$

代表這個 squash commit 原本是在內容為 3 的基礎上，將它改成 7，但目前 v2.0 branch 上的內容實際是 5。

沒有修改

若： $\text{old} = \text{new}$

代表這個 squash commit 執行前後，該 hunk 的內容沒有改變。

因此不需要進行任何操作，current 也不會改變。

情況一：目前內容正是預期的舊內容

若： $\text{current} = \text{old}$

代表目前內容與這個 patch 預期的起點相同，因此可以正常套用修改。

套用後： $\text{current} \leftarrow \text{new}$

例如：

```
patch:    3 -> 7
current: 3
```

可以正常套用，完成後 current 變成 7。

情況二：目前內容已經是目標內容

若： $\text{current} = \text{new}$

代表 v2.0 branch 上已經存在這個 squash commit 想要產生的結果。

因此不會發生 conflict，也不需要再次修改。

例如：

```
patch:    3 -> 7
current: 7
```

雖然目前內容不是 patch 預期的舊內容 3，但它已經等於目標內容 7，所以可以直接視為這項修改已經存在。

current 維持為 7。

情況三：目前內容既不是舊內容，也不是新內容

若： $\text{current} \neq \text{old}$

且： $\text{current} \neq \text{new}$

代表 v2.0 branch 上的內容與這個 squash commit 預期的內容不同，而且也不是它想要修改成的結果。

此時會發生 conflict，例如：

patch: 3 -> 7

current: 5

目前內容 5 既不是舊內容 3，也不是新內容 7，因此發生 conflict。

解決 hunk h 的 conflict 需要支付 w_h 的成本。

解決 conflict 後，採用 squash commit 的修改結果： $\text{current} \leftarrow \text{new}$

在前面的例子中，解決後 current 會變成 7。

同一個 squash commit 可能同時修改多個 hunks。每個 hunk 是否發生 conflict 都要分別判斷，發生 conflict 的成本也要分別計算。

— 輸入 —

第一行包含三個整數 N , H , K , 分別表示：- feat/fancy branch 上的 commit 數量；- hunk 的數量；- squash 後必須保留的 commit 數量。

第二行包含 H 個整數 U_h ($1 \leq h \leq H$)

其中 U_h 表示 v2.0 branch 上第 h 個 hunk 的最終內容。

第三行包含 H 個整數 w_h ($1 \leq h \leq H$)

其中 w_h 表示解決第 h 個 hunk 的 conflict 所需的成本。

接下來 N 行，第 i ($1 \leq i \leq N$) 行包含兩個整數 x_i 與 v_i 表示第 i 個 commit 將第 x_i 個 hunk 的內容改成 v_i 。

— 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

— 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit i : $v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且 feat/fancy branch 不會將任何 hunk 改回 0。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	8	$N, H \leq 15$
3	8	$K = 1$
4	8	$K = N$
5	10	每個 hunk 最多只被修改一次
6	26	$N \leq 1000$
7	40	無限制

— 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

— 範例輸出 —

```
0
```

— 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

```
commit 0: (0, 0)
commit 1: (1, 0)
commit 2: (1, 2)
commit 3: (2, 2)
commit 4: (2, 4)
commit 5: (3, 4)
```

可以將 commits 分成：

[1] [2, 3, 4, 5]

第一個 squash commit 將 hunk 1 :

0 -> 1

套用到 v2.0 branch 時，hunk 1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的效果為：

hunk 1 : 1 -> 3

hunk 2 : 0 -> 4

此時 hunk 1 的目前內容為 1，可以正常套用。

hunk 2 的目前內容已經是 4，因此也不會發生 conflict。

總成本為 0。

— 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

— 範例輸出 —

```
7
```

— 範例說明 —

因為 $K = 1$ ，兩個 commits 必須全部 squash 在一起。
Squash 後的效果為：

```
0 -> 2
```

但 v2.0 branch 上該 hunk 的內容為 1。
目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，成本為 7。

— 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

— 範例輸出 —

```
0
```

— 範例說明 —

因為 $K = 2$ ，兩個 commits 分別套用。
第一個 commit 的效果為：

```
0 -> 1
```

而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。
第二個 commit 的效果為：

1 -> 2

此時目前內容為 1，可以正常套用。
因此總成本為 0。

— **Note** —

E. 你可以幫我吹口哨嗎?

Problem ID: Can_You_Blow_My_Whistle

Time Limit: 1s

Memory Limit: 512MiB



Figure 1: Can You Blow My Whistle Baby

本題為互動題

國中營比賽當天，中午吃飯的時候，有 n 個小學弟排成一列準備領便當。

小學弟們身上穿著 1 到 n 號的衣服，且每個號碼恰好出現一次。

目前他們站在位置 1 到 n 上，位於位置 i 的學弟是 a_i 。

身為大學長的你，希望他們能依照號碼 1 到 n 的順序從小到大排好（也就是位置 i 要站的是號碼 i ）。

為了節省時間，你決定吹哨子發號施令。每次吹哨，你可以同時指定任意多組學弟互換位置，但為了避免碰撞，每個學弟在同一次吹哨中，最多只能參與一組交換。

身為口哨先輩的你，是否可以計算出最少需要吹幾次哨子才能讓所有學弟排好？

— 互動說明 —

你可以使用以下的函式與系統進行互動:

- `void swap_student(int u, int v);`
 - $1 \leq u, v \leq n, u \neq v$
 - 指定位置 u 與位置 v 的學弟互相交換位置。
- `void blow_whistle();`
 - 吹哨! 系統會在你呼叫這個函數時, 統一執行這一輪的所有的 `swap_student` 的指令。
 - 如果在同一輪中, 有同一個學弟被 `swap_student` 呼叫到兩次以上, 則系統會立即中斷, 並給出 **Wrong Answer(1)** 的結果。

你需要實作以下函式:

- `void solve(int n, std::vector<int> a);`
 - 給你人數 n , 以及初始陣列 a , 你需要在 `solve` 函式中實作吹哨流程, 以完成最終排序。

— 輸入限制 —

- $1 \leq n \leq 10^5$

— 給分方式 —

假如你呼叫 `blow_whistle` 函式的次數超過了 $2 \times n$ 次, 則系統會中斷, 並給出 **Wrong Answer(2)** 的結果。

當你的 `solve` 函式結束時, 系統會檢查學弟們是否已經正確排序。若最終沒有完成排序, 你將獲得該筆測資 **0** 分的結果。

若成功完成排序, 對於每個子任務的分數倍率 T , 將依據你的吹哨次數來決定: 令該測資理論最佳的吹哨次數為 opt , 而你所執行的 `blow_whistle` 總次數為 A : - 若 $A = opt$, 則你獲得完美的解, 倍率 $T = 1.0$ 。 - 若 $A > opt$, 則 $T = \max(0.0, 0.5 - (A - opt) \times 0.02)$ 。

也就是說, 只要你的次數不是最佳解, 你最多只能獲得該子任務 50% 的分數, 且每多吹一次哨子, 將扣除 2% 的分數, 直到歸零為止。對於每個滿分為 P 的子任務, 你獲得的總分即為 $P \times T$ 。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	34	$1 \leq n \leq 2000$
2	51	無額外限制
3	15	保證 $opt = 1$

— 範例輸入 —

— 範例輸出 —

G. Performance Analysis

Problem ID: Compiler

Time Limit: 1s

Memory Limit: 512MiB

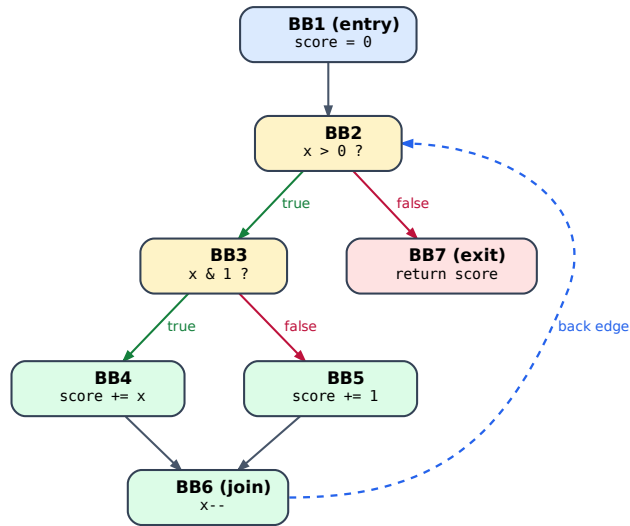


Figure 1: 一個含迴圈與條件分支的 CFG。每個框是一個 basic block； BB_6 是兩條分支的 join point， $BB_6 \rightarrow BB_2$ 是迴圈的 back edge。

tobiichi3227 跟 yushiuan9499 最近在開發 NCC，tobiichi3227 發現它程式執行太慢了，所以想給 NCC 加上一個功能，統計每一條 control-flow edge 會執行幾次。

編譯器將程式用控制流圖 (Control Flow Graph，後面簡稱 CFG) 表示，CFG 上面會有 N 個基本塊 (Basic Block，後面簡稱 BB)，每個 BB 會有幾個有向邊連到其他 BB，代表從 BB_i 可以走到 BB_j ，總共有 M 條有向邊。

程式執行會從入口 1 開始，最後從出口 N 離開結束執行，總共執行 R 次程式。

對於每條邊，編譯器在它所連接的 BB 上插入性能計數器 (BB_i 或 BB_j 都可以，編譯器只會選一個 BB 插入)，因為測量執行次數會造成額外的運算，所以插入每個計數器需要成本 c_e ，執行結束後就能知道該邊通過的次數。

對於沒有裝計數器的邊，可以用 flow conservation 推導計算得知：

- 對於普通節點: $\sum_{e \in \text{in}(v)} f_e = \sum_{e \in \text{out}(v)} f_e \quad (v \neq 1, N)$
- 對於入口: $\sum_{e \in \text{out}(1)} f_e - \sum_{e \in \text{in}(v)} f_e = R$
- 對於出口: $\sum_{e \in \text{in}(N)} f_e - \sum_{e \in \text{out}(N)} f_e = R$

求最小安裝成本，使得執行結束後能夠唯一確定每一條邊通過的次數。

— 輸入 —

第一行會有三個整數 N, M, R ，代表 BB 的數量、邊的數量與執行次數

接下來 M 行會有三個整數 u, v, C ，代表 BB_u 可以走到 BB_v ，且安裝計數器成本為 C

— 輸出 —

最小安裝成本

— 輸入限制 —

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $2M \leq R \leq 10^{18}$
- $1 \leq u, v \leq N$
- $1 \leq C_e \leq 10^9$
- 允許 self-loop 與重邊
- 保證所有 BB 一定有一個 1 到 N 的路徑通過

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	5	$M = N - 1$ ，且第 i 條邊為 $i \rightarrow i + 1$
3	10	$C_e = 1$
4	15	$M \leq 20$
5	15	$M = N$ 且無 self-loop 或重複連接
6	20	每個點雙連通分量皆為單邊分量，或其中每個頂點的分量內度數皆為 2
7	35	無額外限制

— 範例輸入 —

4 4 8
1 2 5
1 3 2
2 4 4
3 4 6

— 範例輸出 —

2

— 範例解釋 —

輸入的 CFG 的結構如下：

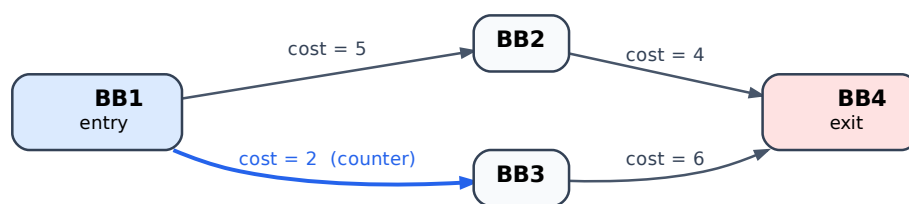


Figure 2: 範例一的 CFG。邊上標示安裝計數器的成本；藍色粗邊表示在 $1 \rightarrow 3$ 安裝計數器。

程式總共執行 8 次。每次執行會選擇下列其中一條路線：

$1 \rightarrow 2 \rightarrow 4$ 或 $1 \rightarrow 3 \rightarrow 4$

可以花費 2，在邊 $1 \rightarrow 3$ 上安裝計數器。

假設程式執行完畢後，計數器顯示邊 $1 \rightarrow 3$ 總共被通過 3 次。因為程式總共執行 8 次，所以邊 $1 \rightarrow 2$ 必定被通過 $8 - 3 = 5$ 次。

每次進入 BB_2 後都會沿著邊 $2 \rightarrow 4$ 離開，因此邊 $2 \rightarrow 4$ 也被通過 5 次。

同理，邊 $3 \rightarrow 4$ 被通過 3 次。

因此，只要在邊 $1 \rightarrow 3$ 上安裝一個計數器，就能確定所有邊的通過次數。

若不安裝任何計數器，則無法判斷 8 次執行中分別有多少次經過 BB_2 或 BB_3 。因此至少需要安裝一個計數器。

所有邊中最低的安裝成本為 2，所以最小總費用為 2。

— 範例輸入 —

4 7 14
1 2 8
2 4 5
1 3 7

3 4 4

2 3 6

2 2 3

1 2 9

— 範例輸出 —

21

— 範例輸入 —

6 6 12

1 2 9

2 3 8

3 4 3

2 4 7

4 5 6

5 6 5

— 範例輸出 —

3

— 範例輸入 —

8 9 18

1 2 10

2 3 8

3 5 2

2 4 6

4 5 5

5 6 7

6 7 4

7 8 3

6 8 9

— 範例輸出 —

5

－ 編譯器小教室：從 C Code 到 CFG －

C 的 if、while、for 等語法適合人類閱讀，但編譯器進行最佳化時，更希望控制流只有少數幾種明確操作：

```
goto L
if condition goto L_true else goto L_false
return value
```

把結構化控制流展開成 label 與 jump 的過程，通常稱為 **lowering**。下文使用的 goto-style 是為了教學設計的 pseudo IR：它和 LLVM IR 一樣用 label 與明確的 branch 描述控制流，但它**不是合法的 LLVM IR**。真正的 LLVM IR 有型別、SSA、phi 等規則，而且使用的 terminator 名稱是 br、ret、switch 等。

概念上的對應關係如下：

```
goto L
    <=> br label %L

if c goto T else goto F
    <=> br i1 %c, label %T, label %F

return x
    <=> ret i32 %x
```

把大括號中的 if、迴圈等結構展開成 label 與 goto，概念上正是 TOJ 756 練習的工作。即使考場沒有網路，也不影響閱讀以下內容；連結只供課後延伸。

Basic Block (BB)

Basic block 是一段**最大且連續**的指令序列，具有以下性質：

- 控制流只會從第一條指令進入，不會跳進 block 中間；
- 除了最後一條指令以外，中途不會跳到別處；
- block 內的指令一旦開始，就會照順序全部執行到 terminator。

在 LLVM-like IR 中，每個 BB 以 label 開始，以一條 terminator 結束。terminator 決定下一個 BB，或直接結束函式。

將一串 goto-style 指令切成 BB 時，可以先找出 **leaders**：

1. 函式的第一條指令是 leader；
2. 任何 jump 的目標 label 是 leader；
3. conditional branch、goto、return 後面的下一條指令若存在，也是新 block 的 leader。

每個 leader 到下一個 leader 之前的所有指令，會形成一個 BB。實作時通常會再刪除 unreachable blocks，並合併不必要的空 blocks。

從 BB 建立 CFG

CFG 中每個節點是一個 BB。若執行完 BB_u 後可能接著執行 BB_v ，就在圖上加入有向邊 $u \rightarrow v$ ：

- goto BB2：一條邊連到 BB_2 ；
- if c goto BB2 else BB3：分別連到 BB_2 與 BB_3 ；
- return：沒有函式內的 successor；
- 若 IR 允許 fall-through，則連到文字順序的下一個 BB。本文會把 jump 全部寫出來，避免隱藏的 fall-through。

因此，goto-style 的 label 先決定 BB 邊界，terminator 再決定 CFG edges。

常見 C 控制結構如何 Lower ?

以下例子都先改寫成 goto-style，再從每個 terminator 直接讀出 CFG。

1. 直線程式

```
x = a + b;
y = x * 2;
return y;
```

沒有 branch，整段就是一個 BB：

```
BB1:
    x = a + b
    y = x * 2
    return y
```

return 是 terminator，因此不能再把其他可執行指令接在它後面。

2. if / else

```
if (x < 0)
    y = -x;
else
    y = x;
use(y);
```

Lowering 後：

```
BB1: if x < 0 goto BB2 else goto BB3
BB2: y = -x; goto BB4
BB3: y = x;  goto BB4
BB4: use(y); ...
```

CFG edges 為 $1 \rightarrow 2$ 、 $1 \rightarrow 3$ 、 $2 \rightarrow 4$ 、 $3 \rightarrow 4$ 。BB₄ 是兩條分支重新會合的 **join point**。若原始程式沒有 else，則 BB₁ 的 false edge 可以直接連到 join point。

3. while

```
while (x > 0) {  
    sum += x;  
    --x;  
}  
use(sum);
```

Lowering 後：

```
BB1:  
    goto BB2  
  
BB2:  
    if x > 0 goto BB3 else goto BB4  
  
BB3:  
    sum += x  
    --x  
    goto BB2  
  
BB4:  
    use(sum)  
    ...
```

BB_2 是 loop header，每次迭代前都檢查條件。3 $\rightarrow$ 2 讓 CFG 形成 cycle；2 $\rightarrow$ 4 則是離開迴圈的 exit edge。

4. do / while

```
do {  
    read_next();  
} while (!ok);  
use_data();
```

Lowering 後：

BB1:

```
    read_next()  
    goto BB2
```

BB2:

```
    if !ok goto BB1 else goto BB3
```

BB3:

```
    use_data()  
    ...
```

條件放在 body 後方，所以 BB_1 至少會執行一次；這就是它和 while CFG 的主要差異。

5. for、break 與 continue

```
for (int i = 0; i < n; ++i) {  
    if (a[i] < 0) continue;  
    if (a[i] == 0) break;  
    sum += a[i];  
}  
use(sum);
```

for (init; cond; step) body 可以拆成 init、condition、body、step 與 exit：

```
BB1: i = 0; goto BB2  
BB2: if i < n      goto BB3 else goto BB7  
BB3: if a[i] < 0   goto BB6 else goto BB4  
BB4: if a[i] == 0  goto BB7 else goto BB5  
BB5: sum += a[i]; goto BB6  
BB6: ++i; goto BB2  
BB7: use(sum); ...
```

continue 必須跳到 step 的 BB_6 ，不能直接跳回 condition，否則會漏掉 ++i。break 則直接跳到 loop exit BB_7 。

6. Short-circuit : && 與 ||

```
if (a != 0 && b / a > 2)
    hit();
next();
```

&& 只有在左側為 true 時才會計算右側，所以兩個判斷通常位於不同 BB：

```
BB1:
    if a != 0 goto BB2 else goto BB4

BB2:
    if b / a > 2 goto BB3 else goto BB4

BB3:
    hit()
    goto BB4

BB4:
    next()
    ...
```

這張 CFG 也解釋了為什麼 $a == 0$ 時不會計算 b / a 。對 $a || b$ 而言則相反：左側為 true 時直接走 true edge，只有左側為 false 才計算右側。

7. switch

```
switch (op) {  
case 0: y = 10; break;  
case 1: y = 20; break;  
default: y = -1;  
}  
use(y);
```

最單純的 lowering 是一串比較：

```
BB1: if op == 0 goto BB3 else goto BB2  
BB2: if op == 1 goto BB4 else goto BB5  
BB3: y = 10; goto BB6  
BB4: y = 20; goto BB6  
BB5: y = -1; goto BB6  
BB6: use(y); ...
```

真正的編譯器也可能保留多路 switch terminator，或產生 jump table。無論機器碼如何實作，CFG 的 dispatch block 都有多個可能 successors，各個 case 最後再連到 join point。

8. 提早 return 每個 return 都會立刻結束目前 BB，而且在函式內沒有 successor。若某個分析希望只有一個出口，可以先將回傳值存入暫存變數，再全部跳到共用出口：

```
BB_return_negative:  
    result = -1  
    goto BB_exit  
  
BB_return_answer:  
    result = answer  
    goto BB_exit  
  
BB_exit:  
    return result
```

這是 compiler pass 常見的 CFG normalization，但不是所有 compiler 都必須這樣做。

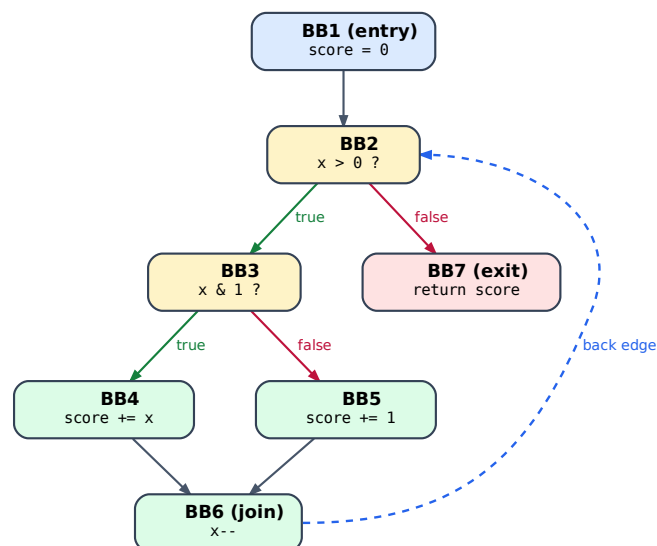
完整範例：從 C、goto-style 到 CFG

題目開頭的 CFG 對應以下 C function：

```
int calc(int x) {
    int score = 0;
    while (x > 0) {
        if (x & 1)
            score += x;
        else
            score += 1;
        --x;
    }
    return score;
}
```

先 lower 成只含明確 jump 的 pseudo IR：

```
BB1: score = 0; goto BB2
BB2: if x > 0 goto BB3 else goto BB7
BB3: if x & 1 goto BB4 else goto BB5
BB4: score += x; goto BB6
BB5: score += 1; goto BB6
BB6: --x; goto BB2
BB7: return score
```



上述 goto-style pseudo IR 建立出的 CFG。

BB_4 與 BB_5 不能合併，因為兩者是不同 branch targets。 BB_6 也必須從 join point 開始，因為它同時有 BB_4 、 BB_5 兩個 predecessors。最後， $BB_6 \rightarrow BB_2$ 回到 loop header，形成迴圈。

支配關係 (Dominance)

在具有入口的 CFG 中，若從 entry 到 v 的**每一條路徑**都一定經過 u ，就稱 u **支配** v ，記為 $u \text{ dom } v$ 。

- 每個節點都支配自己；
- 若 $u \neq v$ 且 u 支配 v ，稱為 strict dominance；
- v 的 **immediate dominator**，記為 $\text{idom}(v)$ ，是最靠近 v 的 strict dominator。

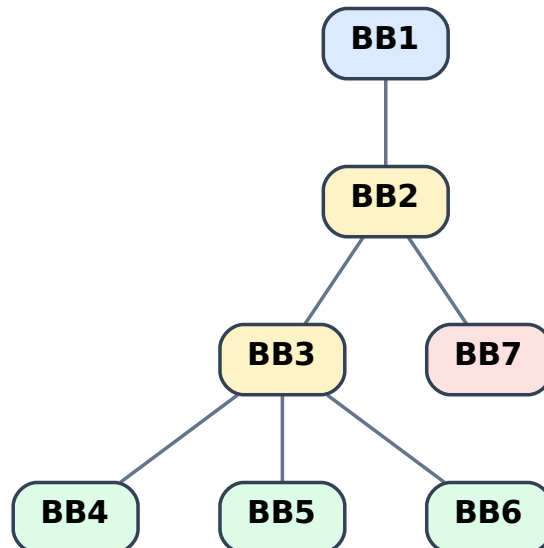
以前面的七個 BB 為例：

節點	Dominators	idom
BB_1	$\{BB_1\}$	無
BB_2	$\{BB_1, BB_2\}$	BB_1
BB_3	$\{BB_1, BB_2, BB_3\}$	BB_2
BB_4	$\{BB_1, BB_2, BB_3, BB_4\}$	BB_3
BB_5	$\{BB_1, BB_2, BB_3, BB_5\}$	BB_3
BB_6	$\{BB_1, BB_2, BB_3, BB_6\}$	BB_3
BB_7	$\{BB_1, BB_2, BB_7\}$	BB_2

BB_4 不支配 BB_6 ，因為可以沿著 $BB_3 \rightarrow BB_5 \rightarrow BB_6$ 到達 BB_6 而不經過 BB_4 。同理， BB_5 也不支配 BB_6 ；但不論走哪一臂都一定先經過 BB_3 ，所以 $\text{idom}(BB_6) = BB_3$ 。

Dominator Tree

把每個非 entry 節點 v 接到 $\text{idom}(v)$ ，就得到 dominator tree：



完整範例的 dominator tree。父節點是子節點的 immediate dominator。

在 dominator tree 中， u 支配 v ，恰好等價於 u 是 v 的 ancestor。注意 dominator-tree edges 代表的是「必經關係」，不一定是原 CFG 中的 control-flow edges。

Dominance 是許多 compiler analyses 的基礎，例如：

- 確認 SSA value 的 definition 是否支配它的 uses；
- 配合 dominance frontier 決定需要放置 phi 的 join points；
- 判斷某段運算能否安全移動；
- 找出 loop header 與 natural loop。

Dominance 與 Loop

對 CFG edge $u \rightarrow h$ ，若 h 支配 u ，則這條邊稱為 **back edge**， h 通常是 loop header。完整範例中， BB_2 支配 BB_6 ，所以 $BB_6 \rightarrow BB_2$ 是 back edge。

這條 back edge 對應的 natural loop 為：

{ BB_2 , BB_3 , BB_4 , BB_5 , BB_6 }

BB_2 是 header， BB_6 是跳回 header 的 latch， $BB_2 \rightarrow BB_7$ 是離開迴圈的 exit edge。
 BB_7 不在 loop 內。

一般有向圖中的 cycle 不一定都具有單一、支配整個 cycle 的 header；這類 CFG 稱為 irreducible。由 while、for 等結構化語法產生的迴圈，通常會形成較容易分析的 reducible CFG。

回到本題

本題已經直接給出 CFG，不需要自行把 C code lowering 成 goto-style。輸入的每個點就是一個 BB，每條有向邊就是一種可能的控制轉移；一次程式執行，就是從 entry 1 沿 CFG 走到 exit N 。性能計數器所測量的，正是這些 control-flow edges 實際被通過的次數。

TODO 競賽名稱
TODO 競賽名稱 試題本

附錄

TODO